

Underwater Glider Internship - Interview Preparation Guide

Newcastle University - Marine Vehicle Lab

Part 1: Behavioral Questions (25 Questions)

Teamwork & Collaboration

1. **Team Dynamics:** You'll be working with a research team on this project. Can you describe a time when you had to collaborate with others on a technical project? How did you handle differences in opinion about the design approach?
2. **Cross-Disciplinary Work:** This project involves mechanical, electrical, and software components. Describe an experience where you worked with people from different technical backgrounds. How did you ensure effective communication?
3. **Feedback Integration:** Tell me about a time when you received critical feedback on your technical work. How did you respond and what did you learn from it?
4. **Supporting Others:** Have you ever helped a teammate who was struggling with a technical concept or task? How did you approach teaching or mentoring them?
5. **Conflict Resolution:** Describe a situation where you had a technical disagreement with a team member. How did you work toward a resolution?

Self-Motivation & Initiative

6. **Independent Projects:** This role requires a "self-motivated" intern. Tell me about a time when you took initiative on a project without being explicitly told what to do. What drove you to take that action?
7. **Going Beyond Requirements:** Describe a project where you went above and beyond the basic requirements. What motivated you to do more?
8. **Learning Drive:** What's the most recent technical skill or concept you taught yourself? What was your learning process?
9. **Problem Identification:** Tell me about a time when you identified a problem that others hadn't noticed. How did you bring it to attention and what was the outcome?

10. **Research Initiative:** Have you ever pursued additional research or reading beyond what was required for a course? What sparked your interest?

Problem-Solving & Adaptability

11. **Unexpected Challenges:** Describe a situation where you encountered an unexpected technical problem while working on a project with a tight deadline. How did you approach solving it?
12. **Resource Constraints:** Tell me about a time when you had to complete a project with limited resources (tools, time, information). How did you adapt?
13. **Failed Approach:** Describe a time when your initial technical approach didn't work. How did you recognize it wasn't working and what did you do next?
14. **Debugging Experience:** Walk me through your process when you encounter a bug that you can't immediately solve. What steps do you take?
15. **Changing Requirements:** Have you ever had project requirements change midway through? How did you handle the pivot?

Learning & Development

16. **New Technologies:** The role involves working with underwater autonomous systems, which may be new to you. Can you give an example of when you had to quickly learn a new technology or skill? What was your approach?
17. **Mistakes & Growth:** Tell me about a significant mistake you made on a technical project. What did you learn and how have you applied that lesson since?
18. **Technical Depth:** Describe a time when you had to dive deep into understanding a complex technical concept. How did you break it down?
19. **Knowledge Sharing:** How do you stay current with developments in your field? Can you give an example of something new you learned recently and how you might apply it?
20. **Seeking Help:** Describe a situation where you needed to ask for help on a technical problem. How did you decide when to ask, and who did you ask?

Time Management & Organization

21. **Multiple Priorities:** This internship is only 6 hours per week over several months. How would you balance this commitment with your academic coursework and other responsibilities?

22. **Project Planning:** Describe how you typically plan and organize a technical project from start to finish. What tools or methods do you use?
 23. **Meeting Deadlines:** Tell me about a time when you had multiple deadlines approaching simultaneously. How did you prioritize and manage your time?
 24. **Long-term Projects:** This internship runs from February to July. How do you maintain motivation and momentum on longer-term projects?
 25. **Documentation Habits:** How do you document your work as you go? Can you give an example of when good documentation practices helped you or your team?
-

Part 2: Technical Questions (30 Questions)

Section A: Python Programming (Control Software & Data Visualization)

Question 1: Basic Control Logic Error

```
def control_buoyancy(current_depth, target_depth):  
    if current_depth < target_depth:  
        return "increase_buoyancy"  
    else:  
        return "decrease_buoyancy"
```

```
# Mission parameters  
target = 50.0 # meters  
current = 49.9 # meters
```

```
action = control_buoyancy(current, target)  
print(f"Action: {action}")
```

What's the problem with this control logic for an underwater glider?
How would you improve it?

Question 2: Data Visualization Enhancement

```
import matplotlib.pyplot as plt
```

```
def plot_glider_path(timestamps, depths):  
    plt.plot(timestamps, depths)  
    plt.show()
```

```
# Sample mission data  
times = [0, 60, 120, 180, 240] # seconds
```

```
depths = [0, -10, -20, -15, -5] # meters
```

```
plot_glider_path(times, depths)
```

What essential information is missing from this visualization? How would you enhance it for mission monitoring?

Question 3: Sensor Data Processing

```
def read_depth_sensor():
    raw_value = sensor.read()
    depth = raw_value * 0.1 # Convert to meters
    return depth

def main_loop():
    while True:
        depth = read_depth_sensor()
        print(f"Depth: {depth}m")
        time.sleep(0.1)
```

What problems might occur with this sensor reading approach in a real underwater environment? How would you make it more robust?

Question 4: Mission Planning System

```
def plan_mission(waypoints):
    for waypoint in waypoints:
        navigate_to(waypoint)

waypoints = [(0, 0), (100, -20), (200, -40), (300, -20)]
plan_mission(waypoints)
```

This is a simplified mission planner. What critical features are missing for a real underwater glider mission? Design a more comprehensive mission planning function.

Question 5: Real-time Data Logging

```
def log_telemetry(depth, battery, position):
    with open('mission_log.txt', 'w') as f:
        f.write(f"{depth},{battery},{position}\n")

while mission_active:
    data = get_telemetry()
    log_telemetry(data.depth, data.battery, data.position)
```

What's wrong with this logging approach? How would you design a better telemetry logging system?

Question 6: Design Challenge - Navigation System Design a Python class for a navigation system that: - Tracks the glider's current position (x, y, depth) - Calculates distance to target waypoint - Determines required pitch angle to reach target depth - Estimates time to reach waypoint

What attributes and methods would your class need? Provide a basic structure.

Question 7: Data Structure Selection

```
mission_data = []

def store_reading(timestamp, depth, temp, pressure):
    mission_data.append(str(timestamp) + "," + str(depth) + "," +
                        str(temp) + "," + str(pressure))

def get_depth_at_time(target_time):
    for entry in mission_data:
        parts = entry.split(",")
        if float(parts[0]) == target_time:
            return float(parts[1])
```

What are the problems with this data storage approach? Suggest better data structures and explain why.

Question 8: Error Handling

```
def calculate_glide_angle(depth_change, horizontal_distance):
    angle = math.atan(depth_change / horizontal_distance)
    return math.degrees(angle)

result = calculate_glide_angle(-15, 0)
```

What will happen when this code runs? How would you add proper error handling and validation?

Question 9: Configuration Management

```
# Glider configuration
MAX_DEPTH = 100
MIN_DEPTH = 0
GLIDE_SPEED = 0.5
```

```
BATTERY_CAPACITY = 5000
```

```
def update_config(param, value):  
    # How would you implement this safely?  
    pass
```

Design a configuration management system for the glider that: - Stores configuration parameters - Validates changes before applying them - Allows saving/loading configurations from file - Provides default values

Question 10: Algorithm Design - Path Smoothing

```
waypoints = [  
    (0, 0, 0),  
    (50, 25, -20),  
    (51, 26, -21),  
    (100, 50, -10)  
]
```

The glider receives waypoints but some are very close together, causing jerky motion. Design an algorithm to smooth the path while maintaining the general route. What approach would you use?

Section B: C++ and Arduino Programming

Question 11: Arduino Control System Bug

```
int pumpPin = 9;  
float targetDepth = 10.0;  
float currentDepth = 0.0;  
  
void setup() {  
    pinMode(pumpPin, OUTPUT);  
}  
  
void loop() {  
    currentDepth = readDepthSensor();  
  
    if (currentDepth < targetDepth) {  
        digitalWrite(pumpPin, HIGH);  
    }  
  
    delay(100);  
}
```

What's wrong with this buoyancy control code? Identify multiple issues and propose solutions.

Question 12: Memory Management

```
void processData() {
    float* readings = new float[1000];

    for(int i = 0; i < 1000; i++) {
        readings[i] = analogRead(A0) * 0.1;
    }

    // Process readings
    calculateAverage(readings, 1000);
}

void loop() {
    processData();
    delay(1000);
}
```

What memory issue exists here? How would you fix it for an embedded system with limited RAM?

Question 13: Timing and Control

```
void adjustPitch() {
    servo.write(45);
    delay(5000); // Wait for adjustment
    servo.write(90);
}

void loop() {
    adjustPitch();
    readSensors();
    updateDisplay();
}
```

Why is using delay() problematic in this control loop? What alternative approach would you use?

Question 14: Sensor Filtering

```
float readDepthSensor() {
    int raw = analogRead(DEPTH_PIN);
```

```

    return raw * 0.0489; // Convert to meters
}

void loop() {
    float depth = readDepthSensor();
    Serial.println(depth);
}

```

Sensor readings are noisy. How would you implement a filter to smooth the depth readings? Provide code for a simple moving average filter.

Question 15: PID Controller Implementation

```

float setpoint = 20.0;
float Kp = 1.0;

float pidControl(float current) {
    float error = setpoint - current;
    float output = Kp * error;
    return output;
}

```

This is a P-only controller. Explain what I and D terms would add, and implement a full PID controller for depth control.

Question 16: Serial Communication

```

void loop() {
    if (Serial.available()) {
        String command = Serial.readString();
        processCommand(command);
    }

    // Send telemetry
    Serial.println(getDepth());
    Serial.println(getBattery());
    Serial.println(getPosition());
}

```

What problems might occur with this serial communication approach? How would you design a more robust protocol?

Question 17: State Machine Design Design a state machine for the glider with the following states: - IDLE - DESCENDING - ASCENDING

- SURFACING (for data transmission) - EMERGENCY

What transitions would exist between states? Write the basic structure in C++.

Question 18: Interrupt Handling

```
volatile int emergencyStop = 0;

void setup() {
    pinMode(EMERGENCY_PIN, INPUT);
    attachInterrupt(digitalPinToInterrupt(EMERGENCY_PIN),
                    emergencyHandler, RISING);
}

void emergencyHandler() {
    emergencyStop = 1;
    stopAllMotors();
    Serial.println("Emergency stop activated!");
    saveState();
    delay(100);
}
```

What's dangerous about this interrupt handler? How should interrupt service routines be written?

Question 19: Power Management

```
void loop() {
    readAllSensors();
    updateDisplay();
    logData();
    checkCommunication();
    delay(10);
}
```

This code runs continuously. How would you implement power-saving features for battery-operated underwater missions? Consider sleep modes and sensor duty cycling.

Question 20: Data Type Selection

```
float depth = 0.0;
float temperature = 0.0;
float pressure = 0.0;
float batteryVoltage = 0.0;
```

```

void updateSensors() {
    depth = readDepth();
    temperature = readTemp();
    pressure = readPressure();
    batteryVoltage = readBattery();
}

```

Is float always the best choice for sensor data on Arduino? Discuss pros/cons and when you might use different data types.

Section C: System Design & Architecture

Question 21: Overall System Architecture Design the software architecture for the underwater glider control system. It needs: - Real-time sensor monitoring - Buoyancy control - Navigation - Data logging - Emergency protocols - Communication with surface station

How would you structure this system? What components/modules would you create?

Question 22: Communication Protocol The glider needs to transmit data to a surface station when it surfaces. Design a simple communication protocol that includes: - Position data - Sensor readings - Battery status - Error/status codes

What format would you use? How would you handle transmission errors?

Question 23: Sensor Fusion The glider has multiple sensors: - Pressure sensor (for depth) - IMU (accelerometer & gyroscope for orientation) - GPS (only works at surface) - Compass

How would you combine data from these sensors to get accurate position and orientation? What algorithm or approach would you use?

Question 24: Fail-Safe Design Design fail-safe mechanisms for these critical failures: - Battery critically low - Depth sensor malfunction - Communication loss - Buoyancy system failure

What should the glider do in each scenario?

Question 25: Mission Replay & Analysis After a mission, you need to analyze what happened. Design a system that: - Logs all relevant data during mission - Allows playback/visualization of the mission - Identifies anomalies or unexpected behavior - Generates a mission report

What data would you log? How would you structure it?

Question 26: Calibration System Sensors need calibration. Design a calibration procedure for: - Depth sensor (pressure to depth conversion) - Pitch angle sensor - Battery voltage to percentage

How would you store calibration data? When should recalibration occur?

Question 27: Testing Strategy You can't test the full system underwater every time. How would you: - Test individual components in isolation - Simulate underwater conditions - Create automated tests - Validate control algorithms before deployment

Describe your testing approach.

Question 28: Real-Time Constraints The control loop must run at a consistent rate (e.g., 10 Hz). How would you ensure timing requirements are met when: - Sensors have variable read times - Calculations may take different amounts of time - Communication might block

What design patterns or techniques would you use?

Question 29: Data Compression The glider collects lots of data but has limited transmission bandwidth when surfacing. Design a strategy to: - Prioritize critical data - Compress less critical data - Decide what to transmit vs. store for later retrieval

What compression or summarization techniques would you consider?

Question 30: Version Control & Updates The glider is deployed for months. How would you: - Manage different software versions - Deploy updates to the glider remotely - Ensure updates don't break critical systems - Roll back if an update causes problems

What's your strategy for software maintenance on deployed systems?

Part 3: Sample Answers Guide

Behavioral Questions - Sample Answers

Question 1: Team Dynamics Sample Strong Answer: “In my second year embedded systems module, our team of four had to build an autonomous line-following robot. We disagreed on whether to use proportional control or a more complex PID controller. I advocated for PID as I’d researched its benefits, while two teammates wanted the simpler approach due to time constraints.

Rather than arguing, I suggested we prototype both approaches in parallel over a weekend. I implemented the PID version while they worked on proportional control. We then tested both with the same test track. The PID performed significantly better, but I acknowledged their approach was faster to implement.

We agreed to use PID but adopted their modular code structure, which was cleaner. This experience taught me that demonstrating solutions practically is more effective than theoretical arguments, and that good ideas can come from any approach.”

Why this is strong: - Specific, concrete example with context - Shows conflict resolution through practical demonstration - Demonstrates respect for others’ viewpoints - Shows learning and compromise - Technical detail appropriate for engineering role

Question 6: Independent Projects Sample Strong Answer: “During my electronics lab sessions, I noticed we were spending a lot of time manually recording oscilloscope measurements. Without being asked, I spent a weekend learning how to interface our oscilloscope with Python using PyVISA.

I created a simple script that could automatically capture measurements and generate formatted lab reports. I showed it to my lab partner first, refined it based on their feedback, then shared it with the class via our course forum.

The lab demonstrator was impressed and asked me to present it in the next session. About half the class ended up using it. What drove me was seeing an inefficiency and knowing I had the skills to solve it. The satisfaction came from seeing others benefit from something I created.

This experience showed me the value of taking initiative—sometimes you don’t need permission to make improvements, you just need to identify a problem and solve it.”

Why this is strong: - Shows genuine initiative without being asked - Clear motivation (solving inefficiency) - Shared solution with others - Positive outcome - Reflects on the experience

Question 11: Unexpected Challenges Sample Strong Answer: “In my microcontrollers coursework, two days before submission, I discovered my Arduino was randomly resetting during operation. I’d been working on the project for weeks, and suddenly nothing worked reliably.

My initial panic lasted about 10 minutes, then I approached it systematically. First, I isolated the problem—was it hardware or software? I loaded a simple blink sketch, which worked fine. Then I progressively added back features until the resets appeared.

I discovered that reading multiple analog sensors simultaneously was causing voltage drops that triggered the brownout detector. I hadn’t considered power supply noise. My solution was twofold: add delays between sensor readings to reduce current spikes, and add a bulk capacitor near the Arduino’s power pins.

With the deadline looming, I also documented the issue clearly in my report, explaining what happened and how I solved it. I actually received extra marks for the troubleshooting process. This taught me to always consider power supply design, even in simple projects, and that systematic debugging beats random fixes.”

Why this is strong: - Specific, relatable technical problem - Shows emotional intelligence (acknowledging panic) - Systematic problem-solving approach - Technical detail and learning - Turned problem into positive outcome

Question 16: New Technologies Sample Strong Answer: “Last summer, I decided to learn ROS (Robot Operating System) even though it wasn’t required for any of my modules. I was interested in robotics and knew ROS was industry-standard.

My approach was to set a concrete goal: build a simulated mobile robot that could navigate a maze. I started with official ROS tutorials, but found them quite abstract. So I supplemented with YouTube videos and GitHub examples from actual robotics projects.

I spent about 2-3 hours per week for a month. When I got stuck on tf transforms (coordinate frame management), I joined the ROS Discord community and asked specific questions. The key was asking after I’d attempted to solve it myself—people are more helpful when you show you’ve tried.

Within a month, I had a working simulation. I then presented this as an extra project during a university robotics club session. Three other students wanted to learn ROS after seeing what it could do, so we formed a study group.

This experience taught me that the best way to learn new technology is to have a concrete project, use multiple learning resources, and engage with the community. I’d apply the same approach to learning about underwater glider systems.”

Why this is strong: - Self-directed learning - Clear methodology - Specific challenges and how overcome - Engaging with community - Sharing knowledge with others - Connects to role requirements

Question 21: Multiple Priorities Sample Strong Answer: “I’d use a combination of time-blocking and priority matrices. Currently, I manage 20 hours of lectures, 15 hours of lab work, and 15 hours of independent study weekly, plus a part-time job on weekends.

For the internship, I’d designate specific weekly slots—for example, two 3-hour blocks on Tuesday and Thursday evenings when I don’t have labs. I’d protect these slots as if they were lectures.

I use a combination of Google Calendar for time-blocking and Notion for task management. At the start of each week, I’d review the internship goals and academic deadlines, then prioritize. If there’s a clash—say, a big coursework deadline coinciding with a critical internship milestone—I’d communicate early with the internship supervisor to adjust timelines.

I’ve successfully managed this before. Last semester, I took an extra module while being course representative, which required attending monthly meetings and gathering student feedback. The key was treating every commitment seriously and communicating when adjustments were needed.

The 6 hours per week structure actually works well for me—it’s enough time to make meaningful progress but allows flexibility to increase hours during quieter academic periods like reading weeks.”

Why this is strong: - Specific, practical strategies - Evidence of current time management - Proactive communication mentioned - Past examples of managing multiple commitments - Positive framing of the internship structure

Technical Questions - Sample Answers

Question 1: Basic Control Logic Error Answer:

Problems identified: 1. **No deadband/tolerance:** The glider will constantly oscillate around the target depth (49.9 vs 50.0). In real systems, you need a tolerance band. 2. **Binary control:** Real buoyancy systems need proportional control—how much to adjust matters, not just direction. 3. **No rate limiting:** Rapid switching can damage actuators and waste energy. 4. **Missing safety checks:** No depth limits, battery checks, or emergency conditions.

Improved version:

```
class BuoyancyController:
    def __init__(self, target_depth):
```

```

self.target_depth = target_depth
self.tolerance = 0.5 # meters
self.max_depth = 100.0
self.min_depth = 0.0
self.last_action_time = 0
self.min_action_interval = 5.0 # seconds

def control_buoyancy(self, current_depth, battery_level):
    # Safety checks first
    if battery_level < 10:
        return "emergency_surface"
    if current_depth > self.max_depth:
        return "emergency_surface"
    if current_depth < self.min_depth:
        return "hold"

    # Check if we're within acceptable range
    error = self.target_depth - current_depth

    if abs(error) < self.tolerance:
        return "hold"

    # Proportional control based on error magnitude
    if error > 0: # Need to go deeper
        if error > 10:
            return "increase_buoyancy_fast"
        else:
            return "increase_buoyancy_slow"
    else: # Need to go shallower
        if abs(error) > 10:
            return "decrease_buoyancy_fast"
        else:
            return "decrease_buoyancy_slow"

```

Key improvements: - Tolerance band prevents oscillation - Proportional control (fast/slow) based on error magnitude - Safety checks for battery and depth limits - Emergency procedures - Rate limiting capability (framework for minimum action intervals)

Question 6: Design Challenge - Navigation System Answer:

```

import math
from datetime import datetime
from dataclasses import dataclass

```

```

@dataclass
class Position:
    """Represents a 3D position"""
    x: float # meters, east
    y: float # meters, north
    depth: float # meters, positive down

class NavigationSystem:
    """
    Navigation system for underwater glider
    Tracks position, calculates routes, estimates timing
    """

    def __init__(self, initial_position, glide_speed=0.5):
        """
        Args:
            initial_position: Starting Position
            glide_speed: Average glide speed in m/s
        """
        self.current_position = initial_position
        self.glide_speed = glide_speed
        self.waypoints = []
        self.position_history = []
        self.last_update_time = datetime.now()

    def add_waypoint(self, waypoint):
        """Add a waypoint to the mission plan"""
        if not isinstance(waypoint, Position):
            raise TypeError("Waypoint must be a Position object")
        self.waypoints.append(waypoint)

    def update_position(self, new_position):
        """Update current position and log to history"""
        self.position_history.append({
            'position': self.current_position,
            'timestamp': datetime.now()
        })
        self.current_position = new_position
        self.last_update_time = datetime.now()

    def distance_to_waypoint(self, waypoint):
        """
        Calculate 3D Euclidean distance to a waypoint

        Returns:
            float: Distance in meters
        """

```



```

    """
    dx = waypoint.x - self.current_position.x
    dy = waypoint.y - self.current_position.y
    dz = waypoint.depth - self.current_position.depth

    return math.sqrt(dx**2 + dy**2 + dz**2)

def horizontal_distance_to_waypoint(self, waypoint):
    """Calculate horizontal (2D) distance"""
    dx = waypoint.x - self.current_position.x
    dy = waypoint.y - self.current_position.y
    return math.sqrt(dx**2 + dy**2)

def calculate_required_pitch(self, waypoint):
    """
    Calculate pitch angle needed to reach target depth

    Returns:
        float: Pitch angle in degrees (positive = nose down)
    """
    horizontal_dist = self.horizontal_distance_to_waypoint(waypoint)
    depth_change = waypoint.depth - self.current_position.depth

    if horizontal_dist < 0.1: # Too close, vertical only
        return 90.0 if depth_change > 0 else -90.0

    # atan2 handles quadrants correctly
    pitch_rad = math.atan2(depth_change, horizontal_dist)
    pitch_deg = math.degrees(pitch_rad)

    # Limit pitch to realistic values for gliders
    max_pitch = 45.0
    return max(-max_pitch, min(max_pitch, pitch_deg))

def bearing_to_waypoint(self, waypoint):
    """
    Calculate bearing to waypoint

    Returns:
        float: Bearing in degrees (0 = North, 90 = East)
    """
    dx = waypoint.x - self.current_position.x
    dy = waypoint.y - self.current_position.y

    bearing_rad = math.atan2(dy, dx)
    bearing_deg = math.degrees(bearing_rad)

```

```

        # Normalize to 0-360
        return (bearing_deg + 360) % 360

def estimate_time_to_waypoint(self, waypoint):
    """
    Estimate time to reach waypoint at average glide speed

    Returns:
        float: Estimated time in seconds
    """
    distance = self.distance_to_waypoint(waypoint)
    return distance / self.glide_speed

def get_next_waypoint(self):
    """
    Get the next waypoint in the mission

    Returns:
        Position or None: Next waypoint or None if no waypoints remain
    """
    if not self.waypoints:
        return None
    return self.waypoints[0]

def waypoint_reached(self, tolerance=5.0):
    """
    Check if current waypoint has been reached

    Args:
        tolerance: Distance threshold in meters

    Returns:
        bool: True if waypoint reached
    """
    if not self.waypoints:
        return False

    distance = self.distance_to_waypoint(self.waypoints[0])

    if distance < tolerance:
        # Remove reached waypoint
        self.waypoints.pop(0)
        return True
    return False

```

```

def get_mission_status(self):
    """
    Get current mission status summary

    Returns:
        dict: Status information
    """
    if not self.waypoints:
        return {
            'waypoints_remaining': 0,
            'mission_complete': True
        }

    next_wp = self.waypoints[0]

    return {
        'current_position': self.current_position,
        'waypoints_remaining': len(self.waypoints),
        'next_waypoint': next_wp,
        'distance_to_next': self.distance_to_waypoint(next_wp),
        'bearing_to_next': self.bearing_to_waypoint(next_wp),
        'required_pitch': self.calculate_required_pitch(next_wp),
        'eta_seconds': self.estimate_time_to_waypoint(next_wp),
        'mission_complete': False
    }

# Example usage
if __name__ == "__main__":
    # Initialize at surface
    start_pos = Position(x=0, y=0, depth=0)
    nav = NavigationSystem(start_pos, glide_speed=0.5)

    # Add mission waypoints
    nav.add_waypoint(Position(x=100, y=50, depth=20))
    nav.add_waypoint(Position(x=200, y=100, depth=40))
    nav.add_waypoint(Position(x=300, y=150, depth=20))

    # Get status
    status = nav.get_mission_status()
    print(f"Distance to next waypoint: {status['distance_to_next']:.2f}m")
    print(f"Required pitch: {status['required_pitch']:.2f}°")
    print(f"ETA: {status['eta_seconds']/60:.1f} minutes")

```

Key design decisions: 1. **Position dataclass:** Clean representation of 3D coordinates 2. **Position history:** Enables trajectory analysis and debugging

3. **Separate 2D/3D calculations:** Important for glider physics
 4. **Bearing and pitch:** Essential for navigation
 5. **Waypoint management:** Queue system with automatic removal
 6. **Status reporting:** Comprehensive mission overview
 7. **Error handling:** Type checks and edge cases (e.g., zero distance)
 8. **Realistic constraints:** Pitch angle limiting
-

Question 11: Arduino Control System Bug Answer:

Problems identified:

1. **No pump turn-off:** Once activated, pump never turns off—will drain battery and potentially damage system
2. **No deadband:** Glider will oscillate around target depth
3. **Blocking delay:** Prevents responsive control and sensor reading
4. **No feedback monitoring:** Can't tell if pump is achieving desired effect
5. **No safety limits:** No maximum depth, battery check, or timeout
6. **Integer/float mixing:** pumpPin is int but depths are float (minor but shows attention to detail)
7. **No state tracking:** Can't tell what mode the system is in

Improved version:

```
// Pin definitions
const int PUMP_PIN = 9;
const int DEPTH_SENSOR_PIN = A0;
const int BATTERY_SENSOR_PIN = A1;

// Control parameters
const float TARGET_DEPTH = 10.0; // meters
const float DEPTH_TOLERANCE = 0.5; // meters
const float MAX_DEPTH = 50.0;
const float MIN_BATTERY = 3.3; // volts
const unsigned long PUMP_TIMEOUT = 30000; // 30 seconds max continuous run

// State tracking
enum SystemState {
    IDLE,
    DESCENDING,
    ASCENDING,
    EMERGENCY
};

SystemState currentState = IDLE;
float currentDepth = 0.0;
float batteryVoltage = 0.0;
unsigned long pumpStartTime = 0;
```

```

bool pumpActive = false;

// Timing (non-blocking)
unsigned long lastSensorRead = 0;
unsigned long lastControlUpdate = 0;
const unsigned long SENSOR_INTERVAL = 100; // 100ms
const unsigned long CONTROL_INTERVAL = 1000; // 1 second

void setup() {
    pinMode(PUMP_PIN, OUTPUT);
    digitalWrite(PUMP_PIN, LOW);

    pinMode(DEPTH_SENSOR_PIN, INPUT);
    pinMode(BATTERY_SENSOR_PIN, INPUT);

    Serial.begin(9600);
    Serial.println("Glider Control System Initialized");
}

void loop() {
    unsigned long currentTime = millis();

    // Read sensors at regular intervals (non-blocking)
    if (currentTime - lastSensorRead >= SENSOR_INTERVAL) {
        lastSensorRead = currentTime;
        readSensors();
    }

    // Update control at regular intervals
    if (currentTime - lastControlUpdate >= CONTROL_INTERVAL) {
        lastControlUpdate = currentTime;
        updateControl();
    }

    // Safety checks run every loop
    performSafetyChecks();
}

void readSensors() {
    // Read and filter depth
    int rawDepth = analogRead(DEPTH_SENSOR_PIN);
    float newDepth = rawDepth * 0.0489; // Convert to meters

    // Simple low-pass filter
    currentDepth = 0.7 * currentDepth + 0.3 * newDepth;
}

```

```

    // Read battery
    int rawBattery = analogRead(BATTERY_SENSOR_PIN);
    batteryVoltage = rawBattery * (5.0 / 1023.0);
}

void updateControl() {
    // Check current state and take appropriate action
    switch(currentState) {
        case IDLE:
            checkIfNeedToMove();
            break;

        case DESCENDING:
            controlDescent();
            break;

        case ASCENDING:
            controlAscent();
            break;

        case EMERGENCY:
            emergencySurface();
            break;
    }

    // Log status
    logStatus();
}

void checkIfNeedToMove() {
    float error = TARGET_DEPTH - currentDepth;

    if (abs(error) < DEPTH_TOLERANCE) {
        // Within acceptable range
        return;
    }

    if (error > 0) {
        // Need to descend
        currentState = DESCENDING;
        activatePump(true);
    } else {
        // Need to ascend
        currentState = ASCENDING;
        activatePump(false);
    }
}

```

```

}

void controlDescent() {
    float error = TARGET_DEPTH - currentDepth;

    if (error < DEPTH_TOLERANCE) {
        // Reached target
        deactivatePump();
        currentState = IDLE;
        Serial.println("Target depth reached");
    }
}

void controlAscent() {
    float error = currentDepth - TARGET_DEPTH;

    if (error < DEPTH_TOLERANCE) {
        // Reached target
        deactivatePump();
        currentState = IDLE;
        Serial.println("Target depth reached");
    }
}

void emergencySurface() {
    // Full buoyancy - surface immediately
    activatePump(false); // Assuming false = positive buoyancy
    Serial.println("EMERGENCY: Surfacing");
}

void activatePump(bool descend) {
    if (descend) {
        digitalWrite(PUMP_PIN, HIGH); // Negative buoyancy
    } else {
        digitalWrite(PUMP_PIN, LOW); // Positive buoyancy
    }

    pumpActive = true;
    pumpStartTime = millis();
    Serial.println(descend ? "Pump ON: Descending" : "Pump OFF: Ascending");
}

void deactivatePump() {
    digitalWrite(PUMP_PIN, LOW);
    pumpActive = false;
    Serial.println("Pump deactivated");
}

```

```

}

void performSafetyChecks() {
    // Check battery
    if (batteryVoltage < MIN_BATTERY) {
        currentState = EMERGENCY;
        Serial.println("CRITICAL: Low battery");
        return;
    }

    // Check maximum depth
    if (currentDepth > MAX_DEPTH) {
        currentState = EMERGENCY;
        Serial.println("CRITICAL: Max depth exceeded");
        return;
    }

    // Check pump timeout
    if (pumpActive && (millis() - pumpStartTime > PUMP_TIMEOUT)) {
        deactivatePump();
        currentState = IDLE;
        Serial.println("WARNING: Pump timeout - deactivated");
    }
}

void logStatus() {
    Serial.print("State: ");
    Serial.print(currentState);
    Serial.print(" | Depth: ");
    Serial.print(currentDepth, 2);
    Serial.print("m | Target: ");
    Serial.print(TARGET_DEPTH, 2);
    Serial.print("m | Battery: ");
    Serial.print(batteryVoltage, 2);
    Serial.println("V");
}

```

Key improvements: 1. **Pump turns off:** Proper state management 2. **Deadband:** Tolerance prevents oscillation 3. **Non-blocking timing:** Using `millis()` instead of `delay()` 4. **State machine:** Clear system states 5. **Safety checks:** Battery, depth limits, pump timeout 6. **Sensor filtering:** Simple low-pass filter for noisy readings 7. **Logging:** Comprehensive status output 8. **Scalability:** Easy to add more sensors/actuators

Question 15: PID Controller Implementation Answer:

Explanation of PID terms:

- **P (Proportional):** Response proportional to current error. Large errors → large corrections. Can oscillate and never quite reach setpoint (steady-state error).
- **I (Integral):** Accumulates error over time. Eliminates steady-state error by compensating for persistent bias. Can cause overshoot and instability if too large.
- **D (Derivative):** Responds to rate of change of error. Provides damping, reduces overshoot and oscillation. Acts as a “brake” when approaching setpoint.

For depth control: - P: Main driving force - I: Corrects for buoyancy drift, current effects - D: Prevents overshoot when approaching target depth

Full PID implementation:

```
class PIDController {
private:
    float kp;    // Proportional gain
    float ki;    // Integral gain
    float kd;    // Derivative gain

    float setpoint;
    float integralSum;
    float lastError;

    unsigned long lastUpdateTime;

    // Anti-windup limits
    float integralMax;
    float integralMin;

    // Output limits
    float outputMax;
    float outputMin;

public:
    PIDController(float p, float i, float d, float target) {
        kp = p;
        ki = i;
        kd = d;
        setpoint = target;

        integralSum = 0;
    }
};
```

```

    lastError = 0;
    lastUpdateTime = millis();

    // Set reasonable limits
    integralMax = 100;
    integralMin = -100;
    outputMax = 255;    // Max PWM for pump
    outputMin = -255;   // Max reverse
}

void setSetpoint(float target) {
    setpoint = target;
}

void setTunings(float p, float i, float d) {
    kp = p;
    ki = i;
    kd = d;
}

void setOutputLimits(float min, float max) {
    outputMin = min;
    outputMax = max;
}

void setIntegralLimits(float min, float max) {
    integralMin = min;
    integralMax = max;
}

float compute(float currentValue) {
    unsigned long currentTime = millis();
    float deltaTime = (currentTime - lastUpdateTime) / 1000.0;    // Convert to seconds

    if (deltaTime <= 0) {
        return 0;    // Prevent division by zero
    }

    // Calculate error
    float error = setpoint - currentValue;

    // Proportional term
    float pTerm = kp * error;

    // Integral term with anti-windup
    integralSum += error * deltaTime;
}

```

```

    integralSum = constrain(integralSum, integralMin, integralMax);
    float iTerm = ki * integralSum;

    // Derivative term
    float errorRate = (error - lastError) / deltaTime;
    float dTerm = kd * errorRate;

    // Combine terms
    float output = pTerm + iTerm + dTerm;

    // Apply output limits
    output = constrain(output, outputMin, outputMax);

    // Update state for next iteration
    lastError = error;
    lastUpdateTime = currentTime;

    return output;
}

void reset() {
    integralSum = 0;
    lastError = 0;
    lastUpdateTime = millis();
}

// Debugging functions
float getProportional(float error) {
    return kp * error;
}

float getIntegral() {
    return ki * integralSum;
}

float getDerivative(float error, float deltaTime) {
    return kd * ((error - lastError) / deltaTime);
}

void printDebug(float currentValue) {
    float error = setpoint - currentValue;
    Serial.print("Error: "); Serial.print(error, 2);
    Serial.print(" | P: "); Serial.print(kp * error, 2);
    Serial.print(" | I: "); Serial.print(ki * integralSum, 2);
    Serial.print(" | D: "); Serial.print(kd * ((error - lastError) / 0.1), 2);
    Serial.println();
}

```

```

    }
};

// Example usage in depth control
PIDController depthPID(1.5, 0.1, 0.5, 20.0); // Target 20m depth

void setup() {
    Serial.begin(9600);

    // Configure PID
    depthPID.setOutputLimits(-100, 100); // Pump control range
    depthPID.setIntegralLimits(-50, 50); // Prevent integral windup
}

void loop() {
    float currentDepth = readDepthSensor();

    // Get PID output
    float pumpControl = depthPID.compute(currentDepth);

    // Apply to pump
    controlBuoyancyPump(pumpControl);

    // Debug output
    depthPID.printDebug(currentDepth);

    delay(100); // 10 Hz control loop
}

void controlBuoyancyPump(float controlSignal) {
    // Convert PID output to pump commands
    if (abs(controlSignal) < 5) {
        // Within deadband - turn off pump
        digitalWrite(PUMP_PIN, LOW);
    } else if (controlSignal > 0) {
        // Need to descend (increase buoyancy)
        analogWrite(PUMP_PIN, constrain(abs(controlSignal), 0, 255));
    } else {
        // Need to ascend (decrease buoyancy)
        analogWrite(PUMP_PIN_REVERSE, constrain(abs(controlSignal), 0, 255));
    }
}

```

Tuning guidelines for underwater glider:

1. **Start with P-only:** Set $K_i=0$, $K_d=0$, adjust K_p until response is reasonable

2. **Add D:** Reduce oscillation and overshoot
3. **Add I last:** Eliminate steady-state error from buoyancy drift

Typical values for depth control: - Kp: 1.0 - 2.0 (depends on actuator power) - Ki: 0.05 - 0.2 (small to prevent windup) - Kd: 0.3 - 0.8 (damps oscillations)

Important considerations: - **Sample time:** PID needs consistent timing - **Integral windup:** Limit integral term to prevent buildup - **Derivative noise:** Raw derivative amplifies noise; consider filtering - **Output saturation:** Actuators have physical limits - **Bumpless transfer:** Reset integral when changing setpoints

Question 21: Overall System Architecture Answer:

High-Level System Architecture:

GLIDER CONTROL SYSTEM

MISSION CONTROLLER (Main Loop)

- State machine management
- High-level decision making
- Mission sequencing

NAVIGATION
MODULE

BUOYANCY
CONTROL
MODULE

SAFETY
MONITOR

SENSOR FUSION & FILTERING

- Kalman filtering
- Dead reckoning
- Data validation

```

                                HARDWARE ABSTRACTION LAYER
- Sensor drivers
- Actuator interfaces
- Communication protocols

```

```

SENSORS      PUMP      LOGGER      COMMS      DISPLAY
-Depth
-IMU
-GPS
-Battery

```

Detailed Component Design:

```

# File: glider_system.py

from dataclasses import dataclass
from enum import Enum
import logging

# ===== DATA STRUCTURES =====

class MissionState(Enum):
    """System-wide states"""
    INITIALIZING = 0
    IDLE = 1
    DIVING = 2
    ASCENDING = 3
    SURFACING = 4
    TRANSMITTING = 5
    EMERGENCY = 6

@dataclass
class SensorData:
    """Aggregated sensor readings"""
    timestamp: float
    depth: float
    pressure: float
    temperature: float
    pitch: float
    roll: float
    heading: float
    battery_voltage: float

```

```

    gps_lat: float = None
    gps_lon: float = None

@dataclass
class ControlCommand:
    """Commands to actuators"""
    pump_power: float # -100 to 100
    ballast_position: float # 0 to 100
    timestamp: float

# ===== MISSION CONTROLLER =====

class MissionController:
    """
    Top-level controller managing mission execution
    """
    def __init__(self):
        self.state = MissionState.INITIALIZING
        self.navigation = NavigationModule()
        self.buoyancy = BuoyancyController()
        self.safety = SafetyMonitor()
        self.sensor_fusion = SensorFusion()
        self.logger = DataLogger()
        self.comms = CommunicationModule()

    def initialize(self):
        """System startup and self-test"""
        logging.info("Initializing glider systems...")

        # Run self-tests
        if not self.safety.run_diagnostics():
            self.state = MissionState.EMERGENCY
            return False

        # Load mission parameters
        self.navigation.load_mission_plan()

        self.state = MissionState.IDLE
        return True

    def main_loop(self):
        """Main control loop - runs continuously"""
        while True:
            # 1. Collect sensor data
            raw_sensors = self.collect_sensor_data()

```

```

# 2. Fuse and filter
filtered_data = self.sensor_fusion.process(raw_sensors)

# 3. Safety checks (highest priority)
if self.safety.check_critical(filtered_data):
    self.handle_emergency()
    continue

# 4. State-based control
control_cmd = self.execute_state_logic(filtered_data)

# 5. Apply control commands
self.apply_controls(control_cmd)

# 6. Log data
self.logger.log(filtered_data, control_cmd, self.state)

# 7. Update navigation
self.navigation.update_position(filtered_data)

# 8. Check state transitions
self.check_state_transitions(filtered_data)

# Control loop timing
time.sleep(0.1) # 10 Hz

def execute_state_logic(self, data):
    """Execute appropriate logic based on current state"""
    if self.state == MissionState.IDLE:
        return self.idle_logic(data)
    elif self.state == MissionState.DIVING:
        return self.diving_logic(data)
    elif self.state == MissionState.ASCENDING:
        return self.ascending_logic(data)
    elif self.state == MissionState.SURFACING:
        return self.surfacing_logic(data)
    elif self.state == MissionState.TRANSMITTING:
        return self.transmitting_logic(data)
    else:
        return ControlCommand(0, 0, time.time())

def diving_logic(self, data):
    """Control logic during dive phase"""
    target_depth = self.navigation.get_target_depth()
    pitch_target = self.navigation.get_target_pitch()

```



```

        # Use PID for depth control
        pump_cmd = self.buoyancy.control_depth(
            data.depth, target_depth
        )

        return ControlCommand(
            pump_power=pump_cmd,
            ballast_position=50.0, # Centered
            timestamp=time.time()
        )

    def check_state_transitions(self, data):
        """Check if we should transition states"""
        if self.state == MissionState.DIVING:
            if self.navigation.waypoint_reached():
                self.state = MissionState.ASCENDING

        elif self.state == MissionState.SURFACING:
            if data.depth < 0.5: # Near surface
                self.state = MissionState.TRANSMITTING

        # Add more transition logic...

# ===== NAVIGATION MODULE =====

class NavigationModule:
    """
    Handles waypoint management and path planning
    """

    def __init__(self):
        self.waypoints = []
        self.current_position = None
        self.current_waypoint_index = 0

    def load_mission_plan(self, filename="mission.json"):
        """Load waypoints from file"""
        # Load from config file
        pass

    def get_target_depth(self):
        """Get target depth for current waypoint"""
        if not self.waypoints:
            return 0
        return self.waypoints[self.current_waypoint_index].depth

    def get_target_pitch(self):

```

```

        """Calculate required pitch angle"""
        # Calculate based on current position and target
        pass

def waypoint_reached(self, tolerance=5.0):
    """Check if current waypoint reached"""
    if not self.waypoints:
        return True

    target = self.waypoints[self.current_waypoint_index]
    distance = self.calculate_distance(self.current_position, target)

    return distance < tolerance

def update_position(self, sensor_data):
    """Update position estimate from sensors"""
    # Dead reckoning when underwater
    # GPS when at surface
    pass

# ===== BUOYANCY CONTROLLER =====

class BuoyancyController:
    """
    Controls buoyancy system with PID
    """
    def __init__(self):
        self.pid = PIDController(kp=1.5, ki=0.1, kd=0.5)

    def control_depth(self, current_depth, target_depth):
        """
        Calculate pump command to reach target depth

        Returns:
            float: Pump power (-100 to 100)
        """
        self.pid.setpoint = target_depth
        return self.pid.compute(current_depth)

# ===== SAFETY MONITOR =====

class SafetyMonitor:
    """
    Monitors for critical conditions
    """
    def __init__(self):

```

```

        self.max_depth = 100.0
        self.min_battery = 10.0
        self.max_pitch = 45.0
        self.max_roll = 30.0

    def check_critical(self, data):
        """
        Check for critical safety conditions

        Returns:
            bool: True if emergency detected
        """
        if data.depth > self.max_depth:
            logging.critical(f"Max depth exceeded: {data.depth}m")
            return True

        if data.battery_voltage < self.min_battery:
            logging.critical(f"Critical battery: {data.battery_voltage}V")
            return True

        if abs(data.pitch) > self.max_pitch:
            logging.warning(f"Excessive pitch: {data.pitch}°")
            return True

        if abs(data.roll) > self.max_roll:
            logging.warning(f"Excessive roll: {data.roll}°")
            return True

        return False

    def run_diagnostics(self):
        """Run system self-test"""
        # Test sensors, actuators, comms
        return True

# ===== SENSOR FUSION =====

class SensorFusion:
    """
    Combines multiple sensor inputs with filtering
    """
    def __init__(self):
        self.depth_filter = KalmanFilter()
        self.attitude_filter = KalmanFilter()

    def process(self, raw_data):

```

```

    """
    Filter and fuse sensor data

    Returns:
        SensorData: Filtered sensor readings
    """
    # Apply Kalman filtering
    filtered_depth = self.depth_filter.update(raw_data.depth)

    # Validate readings
    validated = self.validate_sensors(raw_data)

    return validated

# ===== DATA LOGGER =====

class DataLogger:
    """
    Logs mission data to SD card
    """
    def __init__(self, log_dir="/data"):
        self.log_dir = log_dir
        self.log_file = None

    def log(self, sensor_data, control_cmd, state):
        """Write timestamped data to log"""
        entry = {
            'timestamp': sensor_data.timestamp,
            'state': state.name,
            'depth': sensor_data.depth,
            'battery': sensor_data.battery_voltage,
            'pump_power': control_cmd.pump_power
            # ... more fields
        }

        # Write to file
        self.write_entry(entry)

# ===== COMMUNICATION MODULE =====

class CommunicationModule:
    """
    Handles surface communication (RF, satellite, etc.)
    """
    def __init__(self):
        self.message_queue = []

```

```
def transmit_status(self):
    """Send status update to base station"""
    # Compress and send data
    pass

def receive_commands(self):
    """Check for incoming commands"""
    # Check for new mission parameters
    pass
```

Key Architecture Principles:

1. **Separation of Concerns:** Each module has a single, clear responsibility
2. **Hardware Abstraction:** Control logic doesn't directly access hardware
3. **State Machine:** Clear system states prevent conflicting behaviors
4. **Safety First:** Safety checks run every loop iteration
5. **Modularity:** Modules can be tested independently
6. **Data Flow:** Unidirectional flow from sensors → processing → control
7. **Logging:** Comprehensive logging for post-mission analysis
8. **Error Handling:** Graceful degradation and emergency protocols

Deployment Strategy: - Main control runs on Arduino/microcontroller - Python used for mission planning and data analysis on companion computer - Communication between systems via serial protocol - Real-time components in C++ for deterministic timing - Non-critical analysis in Python for flexibility

This architecture provides a solid foundation that can be expanded with additional sensors, control strategies, and mission capabilities.

Closing Notes

These questions are designed to assess both your technical competence and your ability to work effectively in a research environment. Remember:

For Behavioral Questions: - Use the STAR method (Situation, Task, Action, Result) - Be specific with examples - Show self-awareness and learning - Demonstrate enthusiasm for the role

For Technical Questions: - Think out loud - show your reasoning - Ask clarifying questions if needed - Consider edge cases and safety - Discuss tradeoffs in your solutions - Don't be afraid to say "I don't know, but here's how I'd find out"

Good luck with your interview!